New Relic APM — Day 3 Lab Guide

Hands-on labs for *Performance optimization and dashboards* · easyTravel and Spring PetClinic on Linux · New Relic Java and infrastructure agents

How to use this guide

Each lab matches a module of the New Relic APM — Day 3 Study Guide: read the module, then do its lab. The labs build on the Day 1 and Day 2 lab system.

Lab	Title	Module	Time
3.1	Diagnose a GC-driven response-time degradation using runtime metrics	Module 7	60 min
3.2	Identify and explain the top slow SQL statement impacting a critical transaction	Module 8	75 min
3.3	Build an on-call command-center dashboard for a microservice application	Module 9	60 min

Before you start

- Days 1 and 2 are complete: `etdc` and `pattern on|off <Name>` work, both easyTravel services report to New Relic, and `~/nr-lab/.env` holds your keys.
- Reset: all problem patterns off, MongoDB running, the Day 2 workflow disabled (so today's experiments do not email you).
- The VM needs about **2 GB more memory and 3 GB more disk** for today's additions (PetClinic, PostgreSQL, the infrastructure agent). Check with `free -h` and `df -h ~`.

```
cd ~/nr-lab
for p in DBSpamming JourneySearchError500 TravellersOptionBox LargeMemoryLeak
GarbageCollectionEvery10Seconds; do pattern off $p; done
etdc ps --format "table {{.Name}}\t{{.Status}}"
free -h | head -2; df -h ~ | tail -1
```

Conventions

- `$` lines run on the lab VM as `trainee`, from `~/nr-lab` unless a step says otherwise.
- **Expected output** pictures show terminal output; **Expected view** pictures are redrawn New Relic screens — your numbers, names and layout will differ. Look for the shape: a rising floor, a scan type, a red billboard.
- Restarting the easyTravel backend resets its problem patterns. Steps that restart it say so.
- JVM metric names differ slightly between agent versions. Lab 3.1 shows how to list the names your agent reports; use those in the queries.

Lab 3.1 — Diagnose a GC-driven response-time degradation using runtime metrics

Goal: the easyTravel backend gets slower and slower, for every transaction, without a deployment or a database change. Add host and container metrics, record the backend's healthy JVM baseline, then use heap, garbage-collection and container metrics to prove that a memory leak — through garbage collection — is driving response time.

Time: 60 minutes. **Module:** 7.

Step 1 — Install the infrastructure agent

Run the New Relic infrastructure agent as a container. The *bundle* image also contains the on-host integrations used in Lab 3.2.

```
set -a; source ~/nr-lab/.env; set +a
mkdir -p ~/nr-lab/infra/integrations.d
docker run -d --name newrelic-infra --restart unless-stopped \
  --network=host --cap-add=SYS_PTRACE --privileged --pid=host --cgroupns=host \
  -v "/:/host:ro" -v "/var/run/docker.sock:/var/run/docker.sock" \
  -v "$HOME/nr-lab/infra/integrations.d:/etc/newrelic-infra/integrations.d:ro" \
  -e NRIA_LICENSE_KEY="$NEW_RELIC_LICENSE_KEY" -e NRIA_DISPLAY_NAME=nr-lab-vm \
  newrelic/infrastructure-bundle:latest
sleep 60; docker ps --filter name=newrelic-infra --format "table {{.Names}}\t{{.Status}}"
```

Lab 3.1 · Step 1 · Infrastructure agent — EXPECTED OUTPUT

```
trainee@nr-lab:~/nr-lab$ set -a; source ~/nr-lab/.env; set +a
trainee@nr-lab:~/nr-lab$ mkdir -p ~/nr-lab/infra/integrations.d
trainee@nr-lab:~/nr-lab$ docker run -d --name newrelic-infra --restart unless-stopped
                         --network=host --cap-add=SYS_PTRACE --privileged --pid=host
                         --cgroupns=host -v "/:/host:ro" -v
                         "/var/run/docker.sock:/var/run/docker.sock" -v "$HOME/nr-lab/infra/
                         integrations.d:/etc/newrelic-infra/integrations.d:ro" -e
                         NRIA_LICENSE_KEY="$NEW_RELIC_LICENSE_KEY" -e
                         NRIA_DISPLAY_NAME=nr-lab-vm newrelic/infrastructure-bundle:latest
Unable to find image 'newrelic/infrastructure-bundle:latest' locally
latest: Pulling from newrelic/infrastructure-bundle
Status: Downloaded newer image for newrelic/infrastructure-bundle:latest
3f1c9a7e52b0d4c1a8e9f60b7d2e4a51c3b8f9e0d7a6c5b4a3f2e1d0c9b8a7f6
trainee@nr-lab:~/nr-lab$ sleep 60; docker ps --filter name=newrelic-infra --format "table
                         {{.Names}}\t{{.Status}}"
NAMES            STATUS
newrelic-infra   Up About a minute
in New Relic: Infrastructure > Hosts shows nr-lab-vm, with its containers
```

Expected output — the infrastructure agent running as a container on the lab VM

Confirm the data arrives:

```
SELECT count(*) FROM SystemSample, ProcessSample, ContainerSample FACET eventType() SINCE 5
minutes ago

SELECT latest(cpuPercent) AS 'cpu %', latest(memoryUsageBytes) / 1e6 AS 'memory MB'
FROM ContainerSample FACET name SINCE 5 minutes ago
```

The license key is passed as an environment variable to keep the lab short. In production, build an image with the key in `newrelic-infra.yml` or use a secret store.

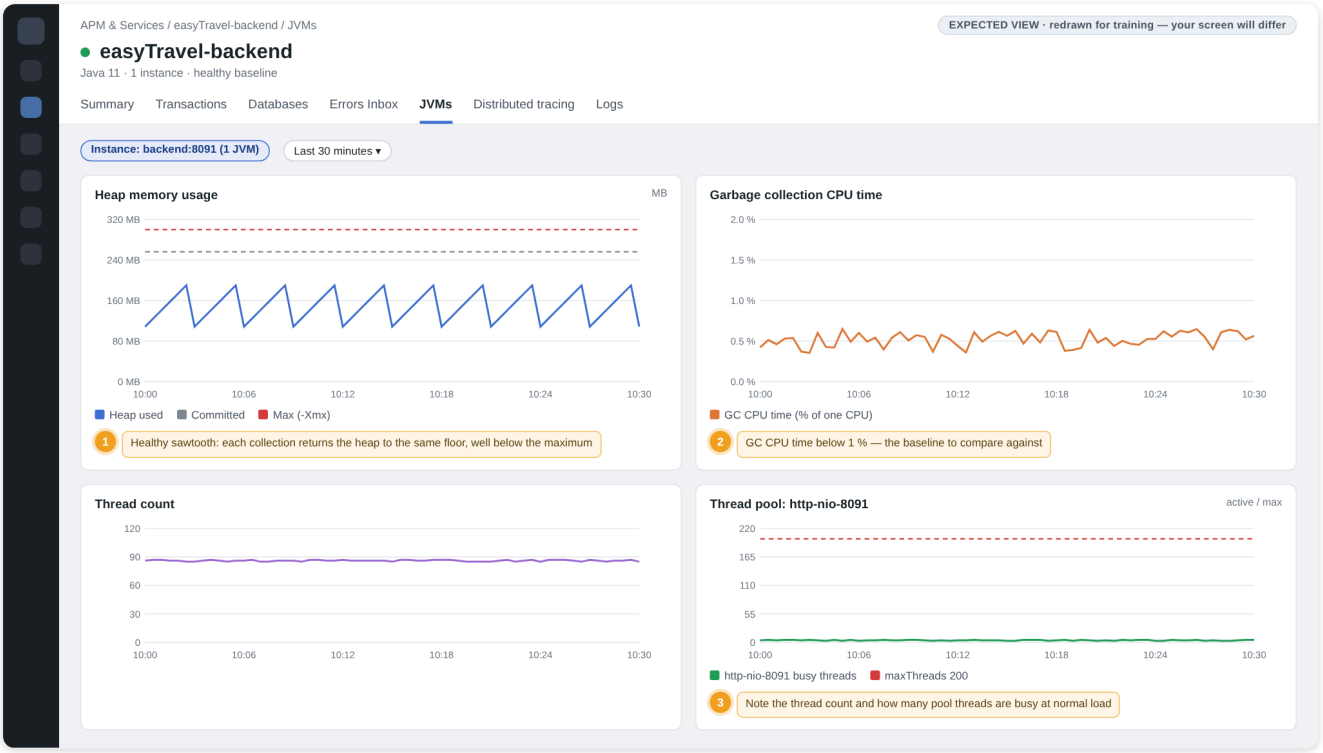
Step 2 — Record the healthy baseline

1. Open **APM & Services → easyTravel-backend → JVMs** (under *More views* in some accounts). Note for the last 30 minutes: **heap used** and **committed**, **GC CPU time**, **thread count**, and the Tomcat **thread pool** *active / max*.
2. List the JVM metric names your agent reports:

```
SELECT uniques(metricTimesliceName) FROM Metric
WHERE appName = 'easyTravel-backend'
  AND (metricTimesliceName LIKE 'Memory/%' OR metricTimesliceName LIKE 'GC/%'
       OR metricTimesliceName LIKE 'Threads/%' OR metricTimesliceName LIKE 'JmxBuiltIn/%')
SINCE 30 minutes ago
```

3. Save the baseline (use the heap and GC names from the list):

```
SELECT average(newrelic.timeslice.value) AS 'heap used (MB)' FROM Metric
WHERE appName = 'easyTravel-backend' AND metricTimesliceName = 'Memory/Heap/Used'
TIMESERIES 1 minute SINCE 30 minutes ago
```



Expected view — healthy JVM: a level sawtooth well under the 300 MB maximum, GC CPU time near zero

Step 3 — Start the leak

`LargeMemoryLeak` makes the business backend keep objects it should release, during location auto-completion. The load generator types destinations; you can speed it up by typing a few letters into the *Where do you want to go?* box on `http://<VM-IP>/` several times.

```
pattern on LargeMemoryLeak
date +%H:%M
```

```
for i in 1 2 3 4 5 6; do docker stats --no-stream backend --format "{{.Name}}  mem
{{.MemUsage}}  cpu {{.CPUPerc}}"; sleep 120; done
```

Lab 3.1 · Step 3 · Start the leak — EXPECTED OUTPUT

```
trainee@nr-lab:~/nr-lab$ pattern on LargeMemoryLeak
LargeMemoryLeak enabled=true -> HTTP 200
trainee@nr-lab:~/nr-lab$ date +%H:%M
10:34
trainee@nr-lab:~/nr-lab$ for i in 1 2 3 4 5 6; do docker stats --no-stream backend --format
                         "{{.Name}}  mem {{.MemUsage}}  cpu {{.CPUPerc}}"; sleep 120; done
backend  mem 412.3MiB / 7.66GiB  cpu 6.12%
backend  mem 448.9MiB / 7.66GiB  cpu 9.85%
backend  mem 476.0MiB / 7.66GiB  cpu 17.40%
backend  mem 489.7MiB / 7.66GiB  cpu 31.26%
backend  mem 494.1MiB / 7.66GiB  cpu 52.83%
backend  mem 495.0MiB / 7.66GiB  cpu 71.09%
memory flattens at the heap limit while CPU keeps rising: the JVM is busy collecting garbage
```

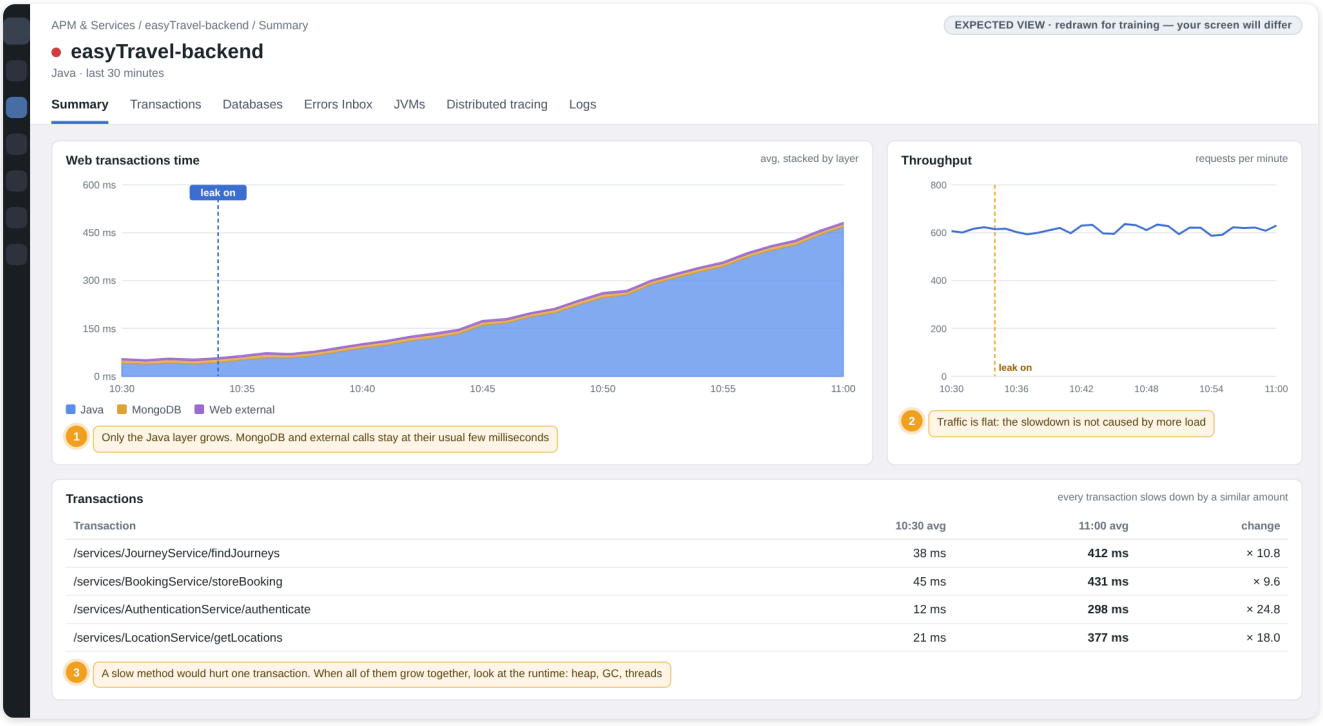
Expected output — the leak enabled; the backend container's memory and CPU climb over ten minutes

> If memory does not climb within 10 minutes, the load may not be using auto-completion. Switch to `MediumMemoryLeak` as well, and keep typing destinations in the browser.

Step 4 — Read the symptoms

Open **easyTravel-backend** → **Summary** for the last 30 minutes:

- **Web transaction time** rises steadily — and the **Java** layer grows, not MongoDB.
- **Throughput** is flat: this is not more traffic.
- Every transaction is affected; no single trace segment explains the extra time.



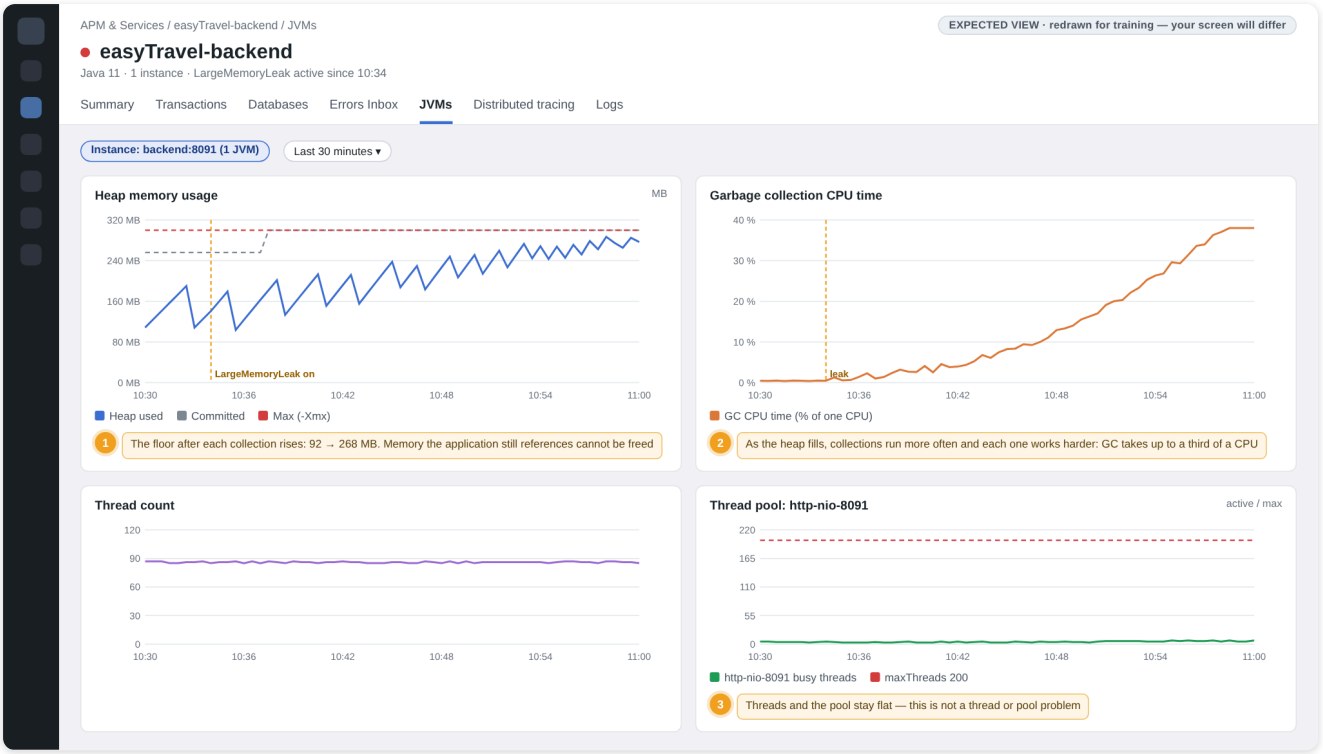
Expected view — backend Summary: response time climbs in the Java layer while throughput stays flat

Question 1. Following the study-guide runbook, which page do you open next, and why?

Step 5 — Read the JVM

Back on the **JVMs** page:

- **Heap used**: each garbage collection frees less — the **floor of the sawtooth rises** towards the maximum.
- **GC CPU time**: rising sharply as the heap fills; the JVM spends more CPU collecting than before.
- **Thread count / pool**: roughly unchanged — not a thread or pool problem.



Expected view — leaking JVM: the floor after each GC rises towards the maximum while GC CPU time climbs

Step 6 — Correlate with NRQL

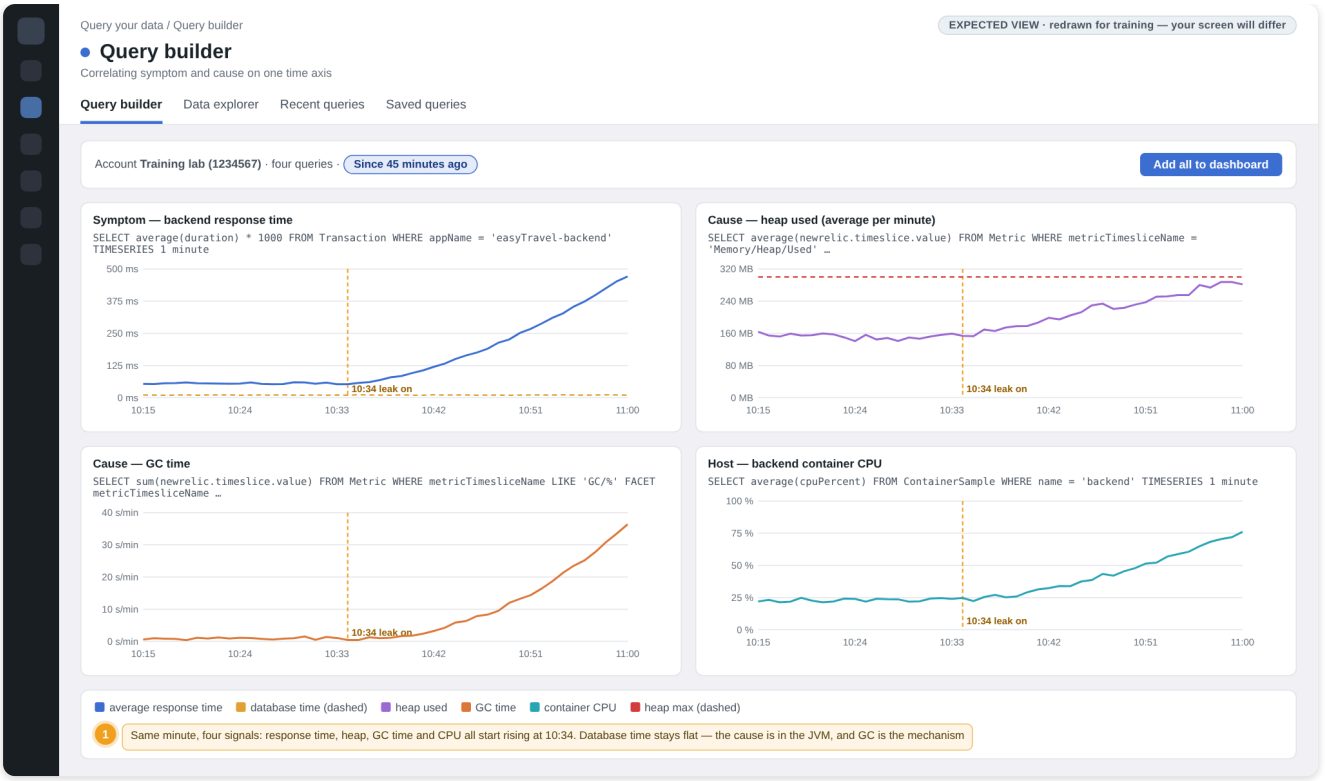
Put the cause and the symptom on one time axis:

```
-- Symptom: backend response time
SELECT average(duration) * 1000 AS 'avg ms' FROM Transaction
WHERE appName = 'easyTravel-backend' TIMESERIES 1 minute SINCE 45 minutes ago

-- Cause: heap used and GC time (names from Step 2)
SELECT average(newrelic.timeslice.value) AS 'heap used (MB)' FROM Metric
WHERE appName = 'easyTravel-backend' AND metricTimesliceName = 'Memory/Heap/Used'
TIMESERIES 1 minute SINCE 45 minutes ago

SELECT sum(newrelic.timeslice.value) FROM Metric
WHERE appName = 'easyTravel-backend' AND metricTimesliceName LIKE 'GC/%'
FACET metricTimesliceName TIMESERIES 1 minute SINCE 45 minutes ago

-- Host view: the backend container
SELECT average(cpuPercent) AS 'cpu %', average(memoryUsageBytes) / 1e6 AS 'memory MB'
FROM ContainerSample WHERE name = 'backend' TIMESERIES 1 minute SINCE 45 minutes ago
```



Expected view — response time, heap used, GC time and container CPU rise together from the moment the leak starts

The evidence lines up: response time, heap floor, GC time and the container's CPU all rise from the same minute; the database layer and throughput do not. The cause is memory held by the application, made visible as garbage-collection time.

Step 7 — Mitigate and verify

A leak's objects are still referenced, so switching the pattern off does not give the memory back. Restart the backend (this also resets its patterns):

```
pattern off LargeMemoryLeak; pattern off MediumMemoryLeak
etdc restart backend && sleep 90
docker stats --no-stream backend --format "{{.Name}}  mem {{.MemUsage}}  cpu {{.CPUPerc}}"
```

Lab 3.1 · Step 7 · Mitigate — EXPECTED OUTPUT

```
trainee@nr-lab:~/nr-lab$ pattern off LargeMemoryLeak; pattern off MediumMemoryLeak
LargeMemoryLeak enabled=false -> HTTP 200
MediumMemoryLeak enabled=false -> HTTP 200
trainee@nr-lab:~/nr-lab$ etdc restart backend && sleep 90
[+] Restarting 1/1
 ✔ Container backend  Started
trainee@nr-lab:~/nr-lab$ docker stats --no-stream backend --format "{{.Name}}  mem {{.MemUsage}}
                         cpu {{.CPUPerc}}"
backend  mem 405.6MiB / 7.66GiB  cpu 5.48%
switching the pattern off does not release leaked objects - only the restart does
```

Expected output — after the restart the backend container is back to its normal memory and CPU

Within 10 minutes the JVMs page shows a level sawtooth again and response time returns to the baseline. In a real incident, take a **heap dump** before restarting (`docker exec backend jcmd 1 GC.heap_dump /tmp/heap.hprof`, assuming the JVM is PID 1) so the developers can find the leaking objects.

Step 8 — A different GC problem: forced full collections

```
pattern on GarbageCollectionEvery10Seconds
date +%H:%M
```

Watch the JVMs page for 5–10 minutes: GC time spikes about every 10 seconds, the heap drops to its floor each time — and the **floor stays level**. Journey searches get slower during each collection. This is not a leak but **explicit full collections** (`System.gc()` in code). The fix is removing the call, or `-XX:+DisableExplicitGC` while it is removed.

```
pattern off GarbageCollectionEvery10Seconds
```

Question 2. How do the heap charts of Step 5 and Step 8 differ, and why does that matter for the fix?

Check your work

- ☐ `SystemSample`, `ProcessSample` and `ContainerSample` arrive from the lab VM.
- ☐ You can show heap, GC time and response time rising together, and the database layer not.
- ☐ The backend was restarted; `LargeMemoryLeak`, `MediumMemoryLeak` and `GarbageCollectionEvery10Seconds` are **off**.

Answers. 1 — The JVMs page: the extra time is in the Java layer across all transactions, with flat throughput, which points to a runtime resource (heap/GC, threads) rather than one slow method or the database. 2 — Leak: the floor after GC rises; the fix is in the code that keeps objects (find it with a heap dump). Forced GC: the floor is level but collections happen far too often; the fix is removing the explicit `System.gc()` call. Raising the heap would only delay the first and would not help the second.

Lab 3.2 — Identify and explain the top slow SQL statement impacting a critical transaction

Goal: the owner search of the PetClinic service is slow. Run PetClinic on PostgreSQL with the Java agent, load realistic data, find the statement that costs the search the most time, read its slow query trace and explain plan, fix it with the right index, prove the improvement, and look at the database from the server side with the PostgreSQL integration.

Time: 75 minutes. **Module:** 8.

Step 1 — Build Spring PetClinic

PetClinic is the Spring team's sample application. It is built in a Maven container, so the VM does not need a JDK.

```
cd ~/nr-lab
git clone --depth 1 https://github.com/spring-projects/spring-petclinic.git petclinic-src
docker run --rm -v "$HOME/nr-lab/petclinic-src":/src -v "$HOME/.m2":/root/.m2 -w /src \
  maven:3.9-eclipse-temurin-17 mvn -q -DskipTests package
cp petclinic-src/target/spring-petclinic-*.jar ~/nr-lab/petclinic.jar
ls -lh ~/nr-lab/petclinic.jar
```

Lab 3.2 · Step 1 · Build Spring PetClinic — EXPECTED OUTPUT

```
trainee@nr-lab:~/nr-lab$ git clone --depth 1
                         https://github.com/spring-projects/spring-petclinic.git
                         petclinic-src
Cloning into 'petclinic-src'...
remote: Enumerating objects: 182, done.
Receiving objects: 100% (182/182), 1.04 MiB | 6.11 MiB/s, done.
trainee@nr-lab:~/nr-lab$ docker run --rm -v "$HOME/nr-lab/petclinic-src":/src -v
                         "$HOME/.m2":/root/.m2 -w /src maven:3.9-eclipse-temurin-17 mvn -q
                         -DskipTests package
(first build: several minutes while Maven downloads dependencies; -q prints nothing on success)
trainee@nr-lab:~/nr-lab$ cp petclinic-src/target/spring-petclinic-*.jar ~/nr-lab/petclinic.jar
trainee@nr-lab:~/nr-lab$ ls -lh ~/nr-lab/petclinic.jar
-rw-r--r-- 1 trainee trainee 58M Oct 29 13:52 /home/trainee/nr-lab/petclinic.jar
```

Expected output — PetClinic built in a Maven container; the application jar copied into the lab folder

The first build downloads dependencies and takes several minutes.

Step 2 — Agent configuration for PetClinic

Lower thresholds than on Day 1, so a few hundred milliseconds are enough for traces, slow query samples and explain plans:

```
mkdir -p ~/nr-lab/agent/petclinic && cp ~/nr-lab/newrelic/newrelic.jar ~/nr-
lab/agent/petclinic/
cat > ~/nr-lab/agent/petclinic/newrelic.yml <<'EOF'
common: &default_settings
  app_name: petclinic
  log_level: info
  distributed_tracing:
    enabled: true
  transaction_tracer:
```

```
    enabled: true
    transaction_threshold: 0.2
    record_sql: obfuscated
    explain_enabled: true
    explain_threshold: 0.1
    stack_trace_threshold: 0.1
  application_logging:
    enabled: true
    forwarding:
      enabled: true
production:
  <<: *default_settings
EOF
```

Step 3 — Start PetClinic and PostgreSQL

```
cat > ~/nr-lab/petclinic-compose.yml <<'EOF'
services:
  petclinic-db:
    image: postgres:17
    container_name: petclinic-db
    environment:
      POSTGRES_USER: petclinic
      POSTGRES_PASSWORD: petclinic
      POSTGRES_DB: petclinic
    command: postgres -c shared_preload_libraries=pg_stat_statements
    ports:
      - "5432:5432"
  petclinic:
    image: eclipse-temurin:17-jre
    container_name: petclinic
    restart: on-failure
    depends_on:
      - petclinic-db
    command: java -javaagent:/opt/newrelic/newrelic.jar -Xmx512m -jar /app/petclinic.jar --
spring.profiles.active=postgres
    environment:
      POSTGRES_URL: jdbc:postgresql://petclinic-db:5432/petclinic
      POSTGRES_USER: petclinic
      POSTGRES_PASS: petclinic
      NEW_RELIC_LICENSE_KEY: ${NEW_RELIC_LICENSE_KEY}
      NEW_RELIC_METADATA_SERVICE_VERSION: "4.0.0"
    volumes:
      - ./petclinic.jar:/app/petclinic.jar:ro
      - ./agent/petclinic:/opt/newrelic
    ports:
      - "8090:8080"
EOF
echo "alias pcdc='docker compose -f ~/nr-lab/petclinic-compose.yml --env-file ~/nr-lab/.env'"
>> ~/.bashrc && source ~/.bashrc
pcdc up -d && sleep 75
curl -s -o /dev/null -w "petclinic: HTTP %{http_code}\n" http://localhost:8090/
```

Lab 3.2 · Step 3 · Start PetClinic and PostgreSQL — EXPECTED OUTPUT

```
trainee@nr-lab:~/nr-lab$ pcdc up -d && sleep 75
[+] Running 3/3
 ✔ Network nr-lab_default   Created
 ✔ Container petclinic-db   Started
 ✔ Container petclinic      Started
trainee@nr-lab:~/nr-lab$ curl -s -o /dev/null -w "petclinic: HTTP %{http_code}\n"
                          http://localhost:8090/
petclinic: HTTP 200
trainee@nr-lab:~/nr-lab$ pcdc logs petclinic | grep -E "New Relic|Started PetClinic" | tail -3
petclinic  | ... com.newrelic INFO: New Relic Agent: Loading configuration file
    "/opt/newrelic/./newrelic.yml"
petclinic  | ... com.newrelic INFO: Reporting to: https://one.newrelic.com/redirect/entity/...
petclinic  | ... Started PetClinicApplication in 9.814 seconds (process running for 14.2)
```

Expected output — PostgreSQL and PetClinic running; PetClinic answers on port 8090

PetClinic uses port 8090 on the VM because easyTravel's nginx already uses 8080. The database credentials are lab-only values.

Step 4 — Load data and traffic

The sample data has ten owners. Production has millions. Add 2 million owners and start a small load script:

```
docker exec -i petclinic-db psql -U petclinic -d petclinic <<'SQL'
INSERT INTO owners (first_name, last_name, address, city, telephone)
SELECT 'Owner' || g, 'Last' || (g % 50000), g || ' Main St.', 'City' || (g % 300), lpad((g %
1000000000)::text, 10, '0')
FROM generate_series(1, 2000000) AS g;
ANALYZE owners;
SELECT count(*) AS owners FROM owners;
SQL

cat > ~/nr-lab/pcload.sh <<'EOF'
#!/usr/bin/env bash
# Owner searches (the critical transaction), owner pages and the vet list - about 3 requests
per second
while true; do
  curl -s -o /dev/null "http://localhost:8090/owners?lastName=Last$((RANDOM % 50000))"
  curl -s -o /dev/null "http://localhost:8090/owners/$((RANDOM % 2000 + 1))"
  curl -s -o /dev/null "http://localhost:8090/vets.html"
  sleep 1
done
EOF
chmod +x ~/nr-lab/pcload.sh
nohup ~/nr-lab/pcload.sh > /dev/null 2>&1 &
```

Lab 3.2 · Step 4 · Data and traffic — EXPECTED OUTPUT

```
trainee@nr-lab:~/nr-lab$ docker exec -i petclinic-db psql -U petclinic -d petclinic <<'SQL'
(INSERT … SELECT … FROM generate_series(1, 2000000); ANALYZE owners; SELECT count(*) …)
INSERT 0 2000000
ANALYZE
 owners
---------
 2000010
(1 row)

trainee@nr-lab:~/nr-lab$ chmod +x ~/nr-lab/pcload.sh
trainee@nr-lab:~/nr-lab$ nohup ~/nr-lab/pcload.sh > /dev/null 2>&1 &
[1] 52190
about one owner search, one owner page and one vet list per second
```

Expected output — 2 million owners inserted and analysed; the load script running

Step 5 — Monitor the database server

Give the infrastructure agent a read-only monitoring user and the PostgreSQL integration configuration:

```
docker exec -i petclinic-db psql -U petclinic -d petclinic <<'SQL'
CREATE USER new_relic WITH PASSWORD 'nr-lab-monitor';
GRANT SELECT ON ALL TABLES IN SCHEMA public TO new_relic;
GRANT pg_read_all_stats TO new_relic;
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
SQL

cat > ~/nr-lab/infra/integrations.d/postgresql-config.yml <<'EOF'
integrations:
  - name: nri-postgresql
    env:
      USERNAME: new_relic
      PASSWORD: nr-lab-monitor
      HOSTNAME: localhost
      PORT: 5432
      DATABASE: petclinic
      COLLECTION_LIST: '["petclinic"]'
      COLLECT_DB_LOCK_METRICS: false
      ENABLE_QUERY_MONITORING: 'true'
      TIMEOUT: 10
    interval: 15s
    labels:
      env: lab
      role: postgresql
    inventory_source: config/postgresql
EOF
chmod 600 ~/nr-lab/infra/integrations.d/postgresql-config.yml
docker restart newrelic-infra && sleep 60
docker logs newrelic-infra 2>&1 | grep -i postgres | tail -3
```

Lab 3.2 · Step 5 · PostgreSQL integration — EXPECTED OUTPUT

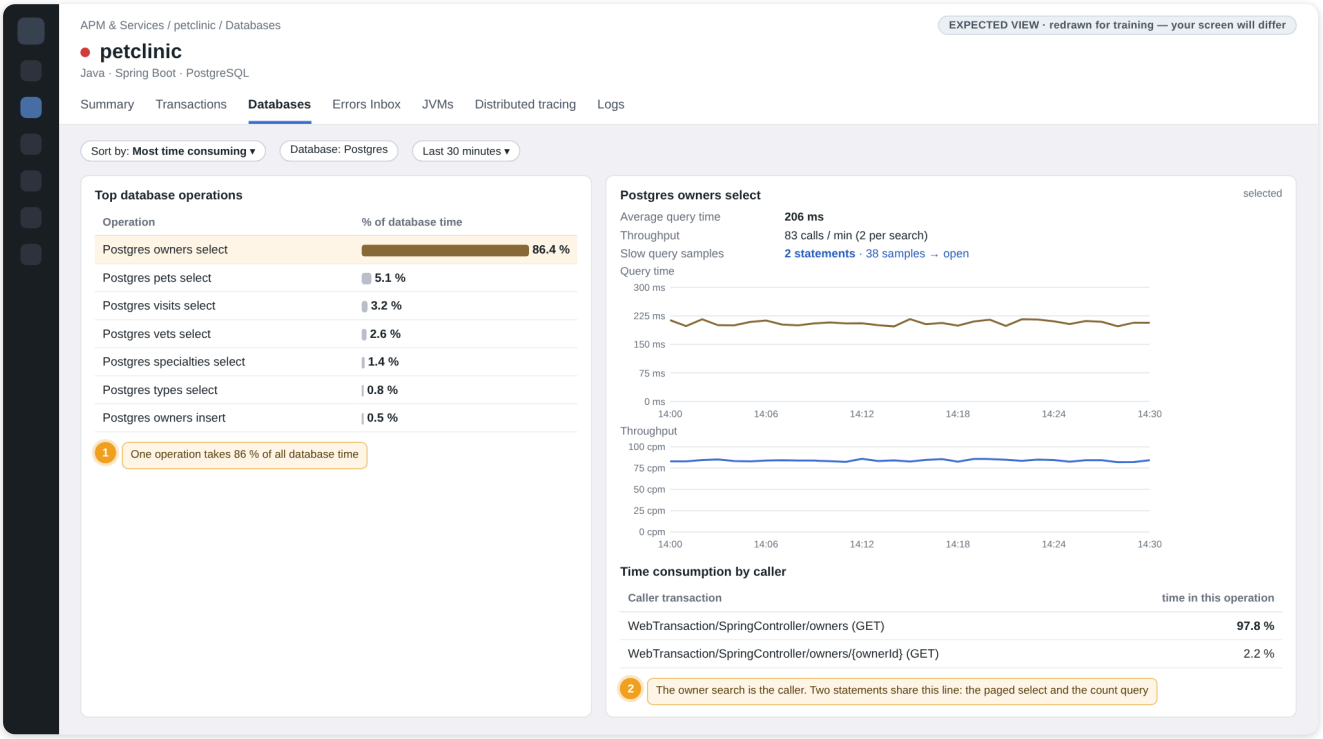
```
trainee@nr-lab:~/nr-lab$ docker exec -i petclinic-db psql -U petclinic -d petclinic <<'SQL'
(CREATE USER new_relic …; GRANT …; CREATE EXTENSION pg_stat_statements)
CREATE ROLE
GRANT
GRANT ROLE
CREATE EXTENSION
trainee@nr-lab:~/nr-lab$ chmod 600 ~/nr-lab/infra/integrations.d/postgresql-config.yml
trainee@nr-lab:~/nr-lab$ docker restart newrelic-infra && sleep 60
newrelic-infra
trainee@nr-lab:~/nr-lab$ docker logs newrelic-infra 2>&1 | grep -i postgres | tail -3
time="2026-10-29T14:04:11Z" level=info msg="Loading integrations file"
    file=/etc/newrelic-infra/integrations.d/postgresql-config.yml
time="2026-10-29T14:04:11Z" level=info msg="Integration added" component=integrations.Manager
    name=nri-postgresql
time="2026-10-29T14:04:27Z" level=info msg="Integration health check finished with success"
    component=integrations.runner.Runner integration_name=nri-postgresql
```

Expected output — the monitoring user created and the PostgreSQL integration loaded by the infrastructure agent

Wait 10 minutes for traffic and slow-query samples to build up.

Step 6 — Find the statement that costs the search the most

1. Open **APM & Services → petclinic → Transactions**, sort by **Most time consuming**: the owner search (named like `WebTransaction/SpringController/owners (GET)`) is at the top.
2. Open **Databases**, sort by **Most time consuming**. `Postgres owners select` takes most of the database time, and the owner search is its main caller. New Relic groups statements by table and operation, so this one line holds two statements: the paged **select** of matching owners and the **count** of all matches that Spring Data runs for the page links.



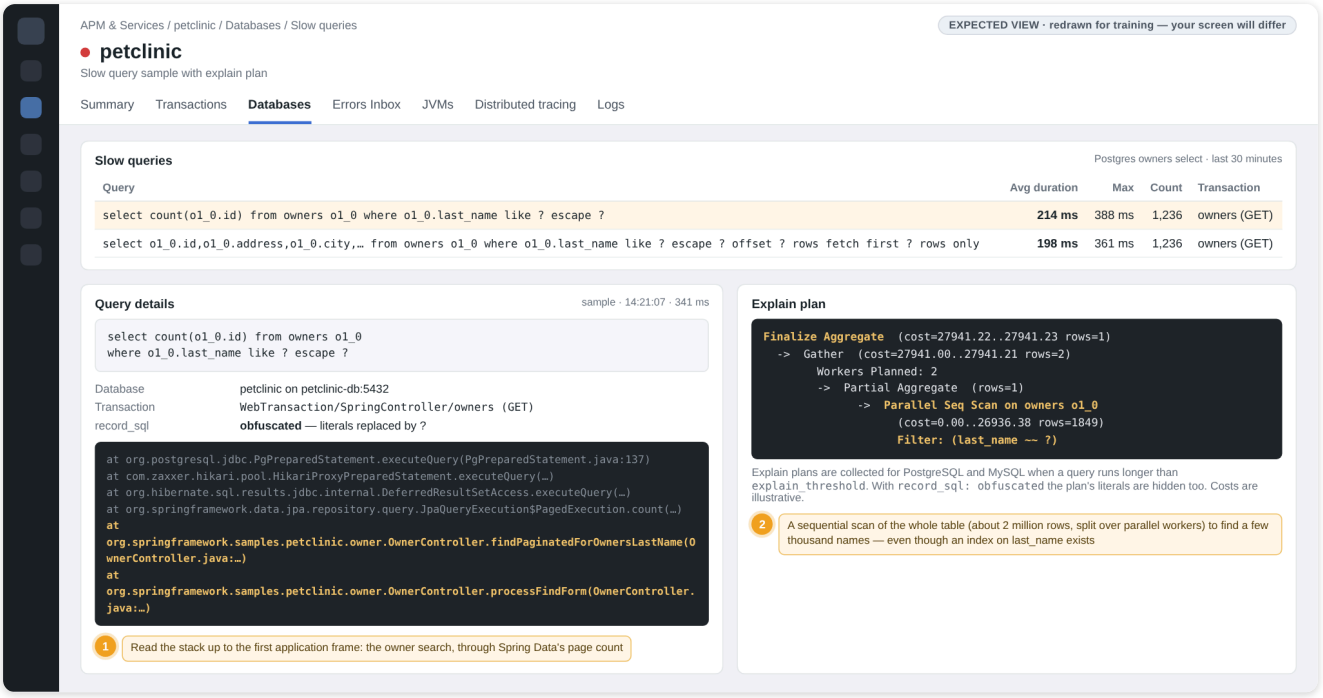
Expected view — the Databases page: the Postgres owners select operation consumes most of the database time and is called by the owner search

```
SELECT average(databaseDuration) / average(duration) * 100 AS 'db %',
       average(databaseCallCount) AS 'db calls', average(duration) * 1000 AS 'avg ms'
FROM Transaction WHERE appName = 'petclinic' FACET name SINCE 15 minutes ago
```

Step 7 — Read the slow query trace and explain plan

On the Databases page, select the `owners select` operation and open a **slow query** sample:

- **SQL** (obfuscated): `select … from owners o1_0 where o1_0.last_name like ? escape ? … fetch first ? rows only`
- **Duration** and **call count**, the **transaction** that ran it, and the **stack trace** up to `OwnerController.processFindForm`.
- **Explain plan**: `Seq Scan on owners` with `Filter: (last_name ~~ …)` over about 2 million rows.



Expected view — slow query sample: obfuscated SQL, the caller stack, and an explain plan showing a sequential scan of 2 million rows

Question 1. The `owners` table *has* an index on `last_name` (`idx_owners_last_name`). Why does the plan still scan the whole table?

Step 8 — Confirm in PostgreSQL

```
docker exec -i petclinic-db psql -U petclinic -d petclinic <<'SQL'
\di idx_owners*
SELECT datcollate FROM pg_database WHERE datname = 'petclinic';
EXPLAIN ANALYZE SELECT count(o.id) FROM owners o WHERE o.last_name LIKE 'Last123%';
SQL
```

Lab 3.2 · Step 8 · Why the index is not used — EXPECTED OUTPUT

```
trainee@nr-lab:~/nr-lab$ docker exec -i petclinic-db psql -U petclinic -d petclinic <<'SQL'
                      List of relations
 Schema |         Name         | Type  |   Owner   | Table
--------+----------------------+-------+-----------+--------
 public | idx_owners_last_name | index | petclinic | owners
 datcollate
------------
 en_US.utf8
                                QUERY PLAN
-------------------------------------------------------------------------------------------
 Finalize Aggregate  (cost=27941.22..27941.23 rows=1 width=8) (actual time=211.4..216.9 rows=1
    loops=1)
   ->  Gather  (actual time=211.1..216.8 rows=3 loops=1)
         Workers Planned: 2   Workers Launched: 2
         ->  Partial Aggregate  (actual time=207.6..207.6 rows=1 loops=3)
               ->  Parallel Seq Scan on owners o  (actual time=0.9..207.3 rows=1480 loops=3)
                     Filter: (last_name ~~ 'Last123%'::text)
                     Rows Removed by Filter: 665190
 Execution Time: 217.1 ms
```

Expected output — the B-tree index exists, the collation is en_US.utf8, and the plan is a sequential scan taking hundreds of milliseconds

With a non-`C` collation, a normal B-tree index cannot answer a `LIKE 'prefix%'` search, so the planner reads every row. The index exists — it is the wrong kind for this query.

Step 9 — Fix: an index the planner can use

```
docker exec -i petclinic-db psql -U petclinic -d petclinic <<'SQL'
CREATE INDEX CONCURRENTLY idx_owners_last_name_pattern ON owners (last_name
text_pattern_ops);
ANALYZE owners;
EXPLAIN ANALYZE SELECT count(o.id) FROM owners o WHERE o.last_name LIKE 'Last123%';
SQL
date +%H:%M
```

Lab 3.2 · Step 9 · The right index — EXPECTED OUTPUT

```
trainee@nr-lab:~/nr-lab$ docker exec -i petclinic-db psql -U petclinic -d petclinic <<'SQL'
CREATE INDEX
ANALYZE
                                QUERY PLAN
-------------------------------------------------------------------------------------------
 Aggregate  (actual time=3.62..3.62 rows=1 loops=1)
   ->  Bitmap Heap Scan on owners o  (actual time=0.71..3.41 rows=4440 loops=1)
         Filter: (last_name ~~ 'Last123%'::text)
         ->  Bitmap Index Scan on idx_owners_last_name_pattern  (actual time=0.52..0.52
    rows=4440 loops=1)
               Index Cond: ((last_name ~>=~ 'Last123'::text) AND (last_name ~<~
    'Last124'::text))
 Execution Time: 3.71 ms
trainee@nr-lab:~/nr-lab$ date +%H:%M
14:38
```

Expected output — the pattern index created; the same query now uses a bitmap index scan and runs in a few milliseconds

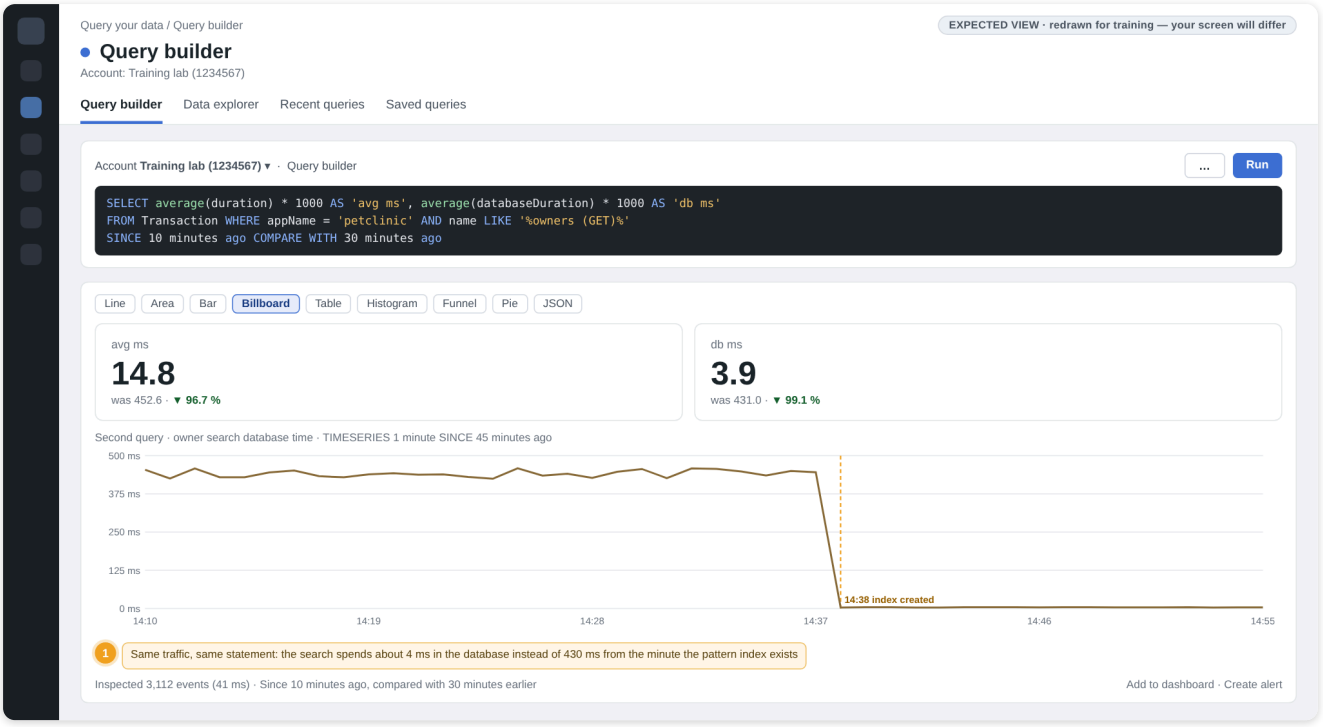
`CREATE INDEX CONCURRENTLY` builds the index without locking out writes — the way it is done on a production table.

Step 10 — Prove it in New Relic

Wait 10 minutes, then compare the search before and after:

```
SELECT average(duration) * 1000 AS 'avg ms', average(databaseDuration) * 1000 AS 'db ms'
FROM Transaction WHERE appName = 'petclinic' AND name LIKE '%owners (GET)%'
SINCE 10 minutes ago COMPARE WITH 30 minutes ago

SELECT average(databaseDuration) * 1000 AS 'db ms' FROM Transaction
WHERE appName = 'petclinic' AND name LIKE '%owners (GET)%' TIMESERIES 1 minute SINCE 45
minutes ago
```



Expected view — owner search database time drops from hundreds of milliseconds to a few at the moment the index was created

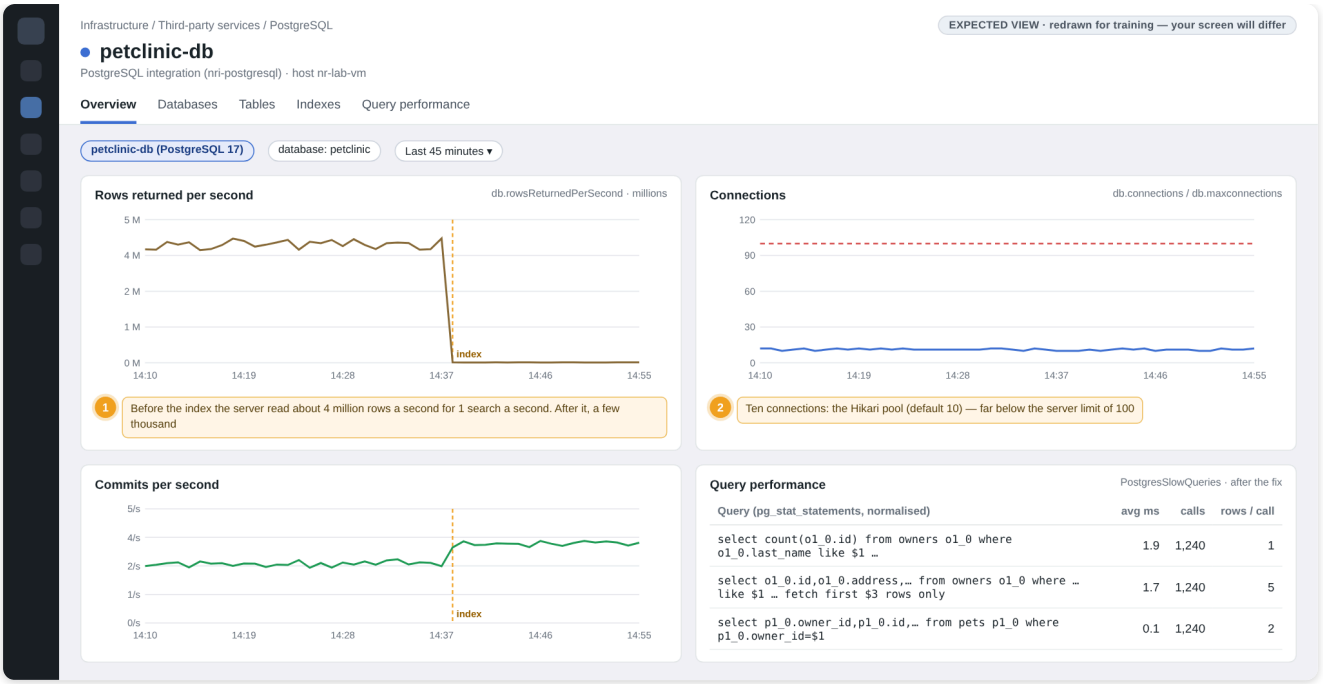
On the Databases page, `owners select` drops down the ranking; new slow query samples no longer appear for it.

Step 11 — The database instance view

Open **Infrastructure → third-party services → PostgreSQL** (or search for the `petclinic-db` / PostgreSQL entity). Check connections, commits and rows returned, and the query-level data:

```
SELECT latest(db.connections) AS 'connections', latest(db.commitsPerSecond) AS 'commits/s',
       latest(db.rowsReturnedPerSecond) AS 'rows returned/s', latest(db.bufferHitsPerSecond)
AS 'buffer hits/s'
FROM PostgresqlDatabaseSample FACET database SINCE 15 minutes ago

SELECT * FROM PostgresSlowQueries SINCE 1 hour ago LIMIT 20
```



Expected view — PostgreSQL instance: connections, commits and rows returned; rows read per second fell sharply after the index

Before the index, *rows returned per second* — rows read by sequential scans — was in the millions: every search read the whole table twice. After it, the server does a fraction of the work for the same traffic. That is the server-side proof of the same fix.

Optional — Connection-pool saturation

Make the pool tiny and the queries slow again, then watch requests queue for connections:

```
docker exec -i petclinic-db psql -U petclinic -d petclinic -c "DROP INDEX CONCURRENTLY
idx_owners_last_name_pattern;"
pcdc stop petclinic
pcdc run -d --name petclinic-small -p 8090:8080 \
  -e SPRING_DATASOURCE_HIKARI_MAXIMUMPOOLSIZE=2 petclinic
for i in 1 2 3 4; do nohup ~/nr-lab/pcload.sh > /dev/null 2>&1 & done
```

Within minutes, response time rises far more than query time: traces show time waiting in the pool (`getConnection`), and the pool metrics (`SELECT uniques(metricTimesliceName) FROM Metric WHERE appName = 'petclinic' AND metricTimesliceName LIKE 'Database Connection/%'`) show *busy* = *max*. Clean up afterwards: `docker rm -f petclinic-small; pcdc up -d petclinic; pkill -f pcload.sh`, recreate the pattern index, and start one `pcload.sh` again.

Check your work

- ☐ You can name the top statement, its transaction and its caller, and explain its plan.
- ☐ The pattern index exists, the plan is an index scan, and New Relic shows the drop.
- ☐ The PostgreSQL entity reports connections and query data.
- ☐ One `pcload.sh` is running (Lab 3.3 needs PetClinic traffic).

Answer. 1 — The database collation is `en_US.utf8`. A normal B-tree index sorts by that collation and cannot be used for `LIKE 'prefix%'`; the planner falls back to a sequential scan. An index with the `text_pattern_ops` operator class compares character by character and can serve prefix searches.

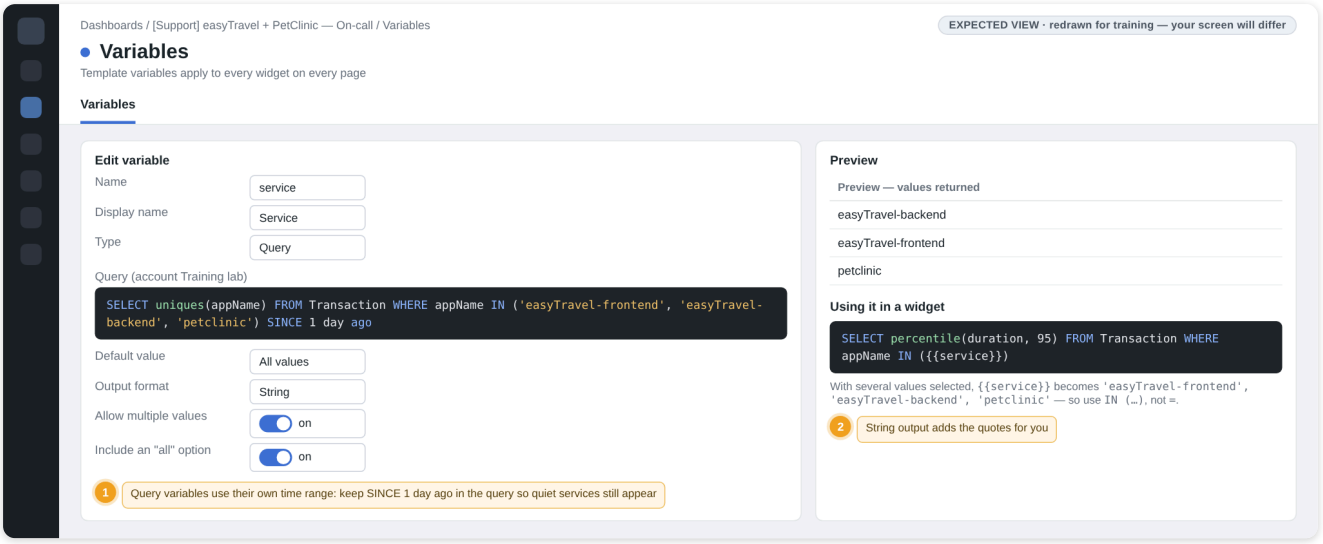
Lab 3.3 — Build an on-call command-center dashboard for a microservice application

Goal: build the dashboard an on-call engineer opens first for the three services — `easyTravel-frontend`, `easyTravel-backend` and `petclinic` — with golden signals, thresholds, a service variable, trends, linked-facet drill-downs, runtime and database health, an SLA page and runbook links; then set its permissions and keep its JSON.

Time: 60 minutes. **Module:** 9.

Step 1 — Create the dashboard and the `service` variable

1. **Dashboards → Create a dashboard → Create a new dashboard.** Name: `[Support] easyTravel + PetClinic — On-call`. Rename the first page `On-call`.
2. In edit mode, **+ Add variable**:
   - Name `service`, title *Service*, type **Query**: `SELECT uniques(appName) FROM Transaction WHERE appName IN ('easyTravel-frontend', 'easyTravel-backend', 'petclinic') SINCE 1 day ago`
   - **Multi-value** on, default: all values, output format **String**.



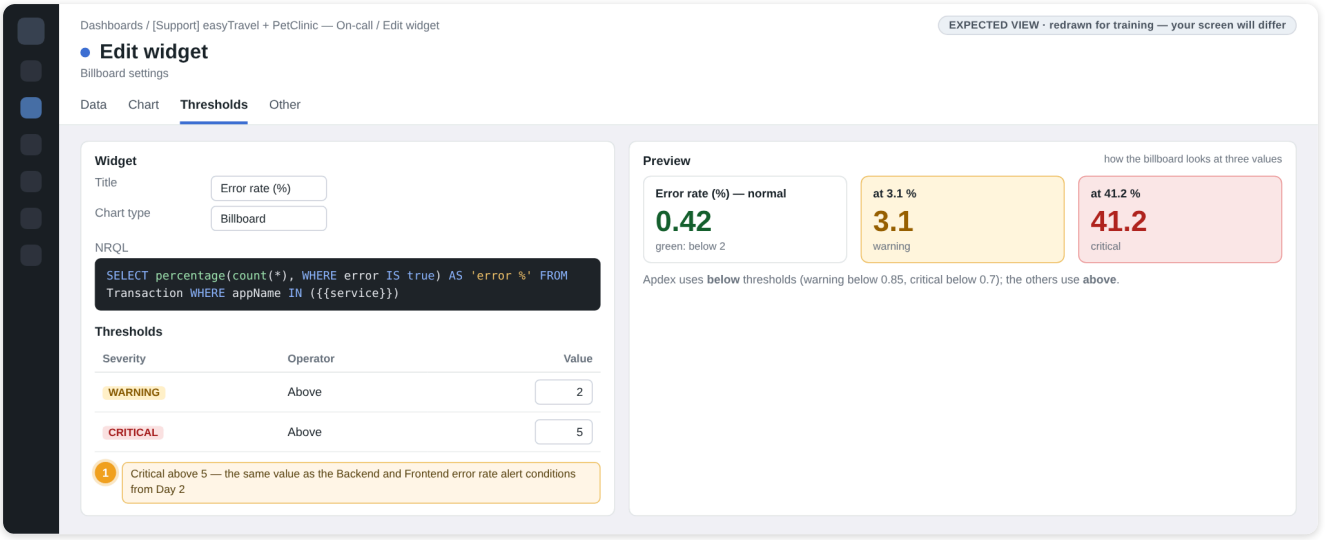
Expected view — the service variable: an NRQL query returning the three services, multi-value, all selected by default

Step 2 — Row 1: golden-signal billboards with thresholds

Add five **billboard** widgets (*Add widget → Add a chart → NRQL*). Set the thresholds in each widget's settings so they match the Day 2 alert conditions.

Title	Query	Warning	Critical
Throughput (rpm)	`SELECT rate(count(*), 1 minute) AS 'rpm' FROM Transaction WHERE appName IN ({{service}})`	—	—
Error rate (%)	`SELECT percentage(count(*), WHERE error IS true) AS 'error %' FROM Transaction WHERE appName IN ({{service}})`	above 2	above 5
p95 latency (s)	`SELECT percentile(duration, 95) AS 'p95 (s)' FROM Transaction WHERE appName IN ({{service}})`	above 1	above 3

Title	Query	Warning	Critical
Apdex	`SELECT apdex(duration, t: 0.5) FROM Transaction WHERE appName IN ({{service}})`	below 0.85	below 0.7
Busiest container CPU (%)	`SELECT max(cpuPercent) AS 'cpu %' FROM ContainerSample WHERE name IN ('frontend', 'backend', 'mongodb', 'petclinic', 'petclinic-db')`	above 70	above 90



Expected view — billboard settings: warning and critical thresholds that match the alert conditions

Step 3 — Row 2: trends

Title	Type	Query
p95 latency by service	Line	`SELECT percentile(duration, 95) FROM Transaction WHERE appName IN ({{service}}) FACET appName TIMESERIES`
Error rate by service	Line	`SELECT percentage(count(*), WHERE error IS true) FROM Transaction WHERE appName IN ({{service}}) FACET appName TIMESERIES`
Throughput vs last week	Line	`SELECT rate(count(*), 1 minute) FROM Transaction WHERE appName IN ({{service}}) TIMESERIES COMPARE WITH 1 week ago`

Step 4 — Row 3: breakdowns with linked facets

Title	Type	Query	Linked facet
Top errors	Table	`SELECT count(*) FROM TransactionError WHERE appName IN ({{service}}) AND error.expected IS NOT true FACET error.class, appName LIMIT 10`	on
Slowest transactions (p95)	Bar	`SELECT percentile(duration, 95) FROM Transaction WHERE appName IN ({{service}}) FACET name LIMIT 10`	—
Database share by service	Bar	`SELECT average(databaseDuration) / average(duration) * 100 AS 'db %' FROM Transaction WHERE appName IN ({{service}}) FACET appName`	on

For the two marked widgets, open the widget settings and turn on **filter the current dashboard** (facet linking). Clicking a bar or a row then filters the page to that value — every widget whose data has that attribute.

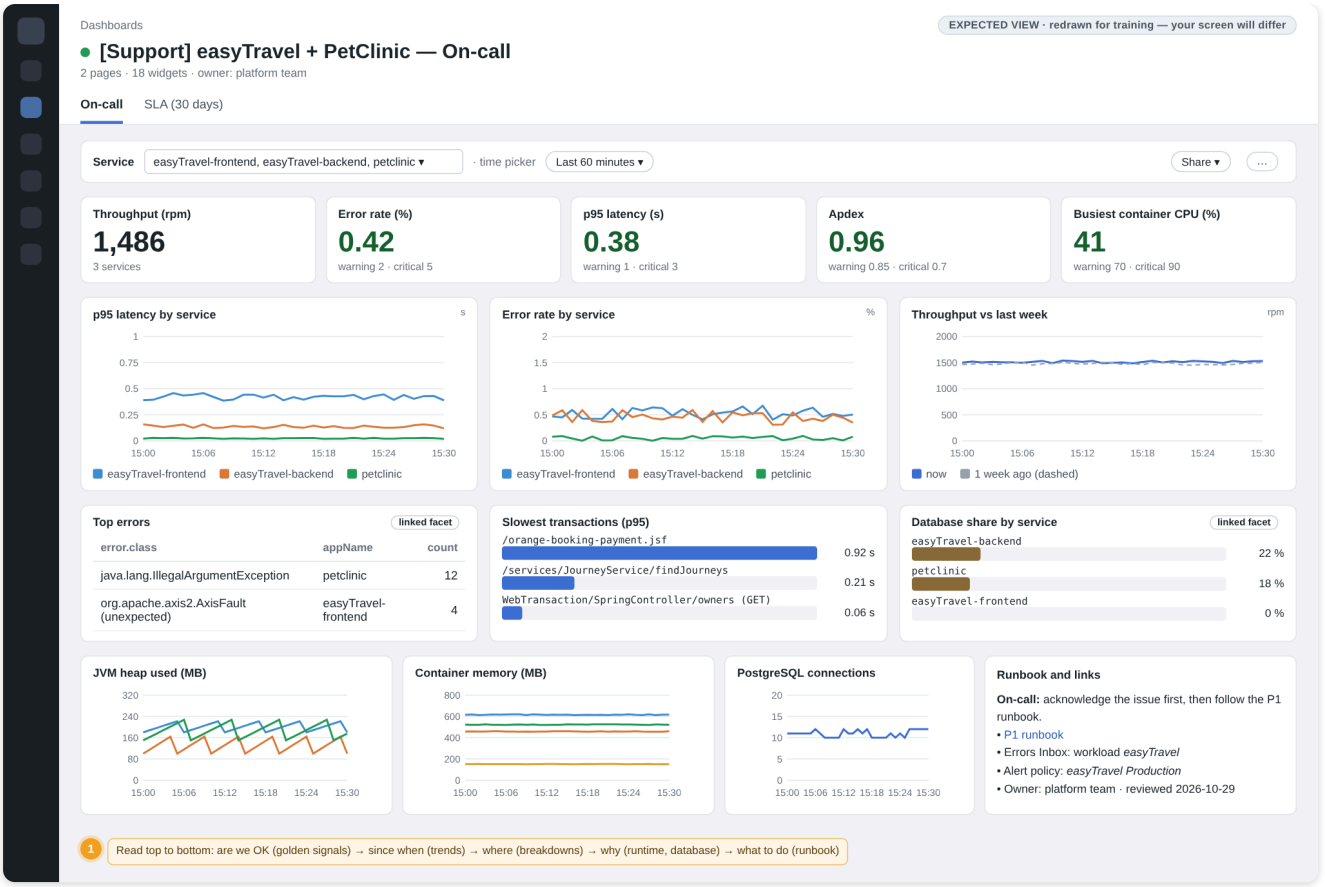
Step 5 — Row 4: runtime, database and dependencies

Title	Type	Query
JVM heap used (MB)	Line	`SELECT average(newrelic.timeslice.value) FROM Metric WHERE appName IN ({{service}}) AND metricTimesliceName = 'Memory/Heap/Used' FACET appName TIMESERIES`
Container memory (MB)	Line	`SELECT average(memoryUsageBytes) / 1e6 FROM ContainerSample WHERE name IN ('frontend', 'backend', 'mongodb', 'petclinic', 'petclinic-db') FACET name TIMESERIES`
PostgreSQL connections	Line	`SELECT latest(db.connections) FROM PostgresqlDatabaseSample TIMESERIES`
Slowest datastore calls	Table	`SELECT average(duration.ms), count(*) FROM Span WHERE appName IN ({{service}}) AND category = 'datastore' FACET appName, name LIMIT 10`

Add a **markdown** widget titled *Runbook and links*:

```
**On-call:** acknowledge the issue first, then follow the P1 runbook.

- [P1 runbook](https://wiki.example.com/runbooks/easytravel-p1)
- Errors Inbox: workload *easyTravel*
- Alert policy: *easyTravel Production*
- Owner: platform team · reviewed 2026-10-29
```

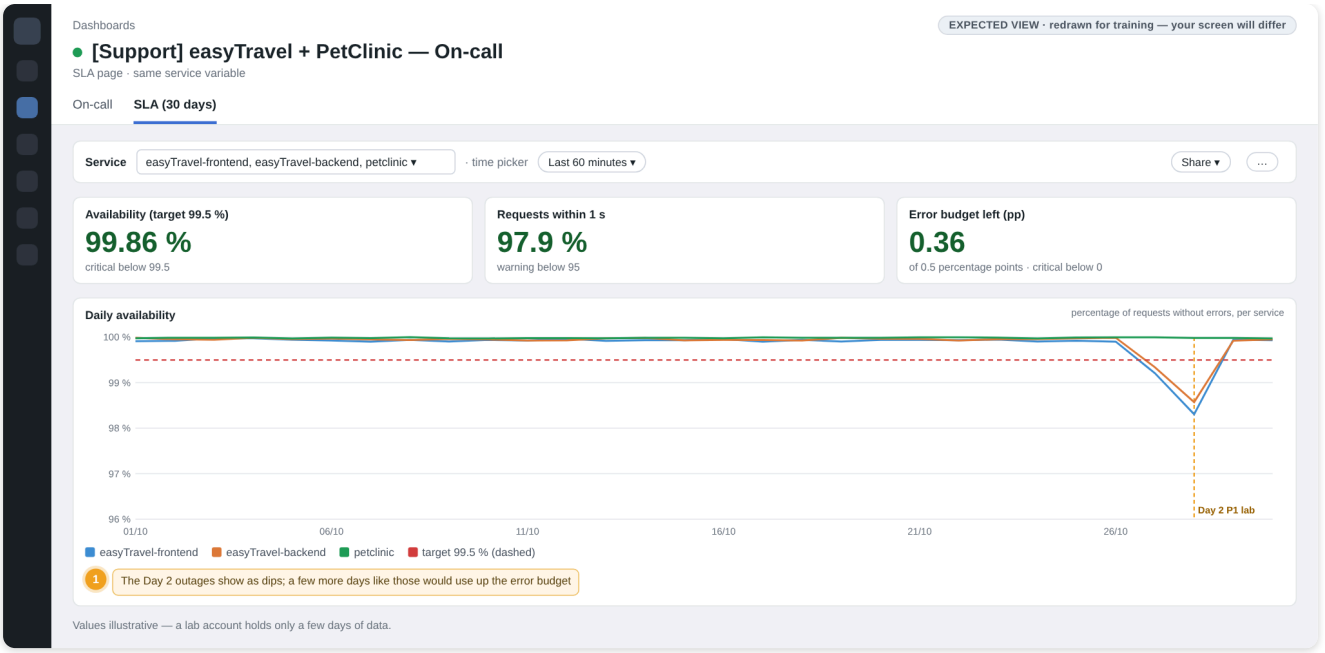


Expected view — the finished On-call page: golden signals, trends, linked-facet breakdowns, runtime and database health, runbook links

Step 6 — Page 2: SLA

Add a page `SLA (30 days)` with these widgets (the variable still applies):

Title	Type	Query	Threshold
Availability (target 99.5 %)	Billboard	`SELECT percentage(count(*), WHERE error IS false) FROM Transaction WHERE appName IN ({{service}}) SINCE 30 days ago`	critical below 99.5
Requests within 1 s	Billboard	`SELECT percentage(count(*), WHERE duration <= 1) FROM Transaction WHERE appName IN ({{service}}) SINCE 30 days ago`	warning below 95
Error budget left (pp)	Billboard	`SELECT 0.5 - percentage(count(*), WHERE error IS true) FROM Transaction WHERE appName IN ({{service}}) SINCE 30 days ago`	critical below 0
Daily availability	Line	`SELECT percentage(count(*), WHERE error IS false) FROM Transaction WHERE appName IN ({{service}}) FACET appName SINCE 30 days ago TIMESERIES 1 day`	—



Expected view — the SLA page: 30-day availability, latency compliance and error budget for the selected services

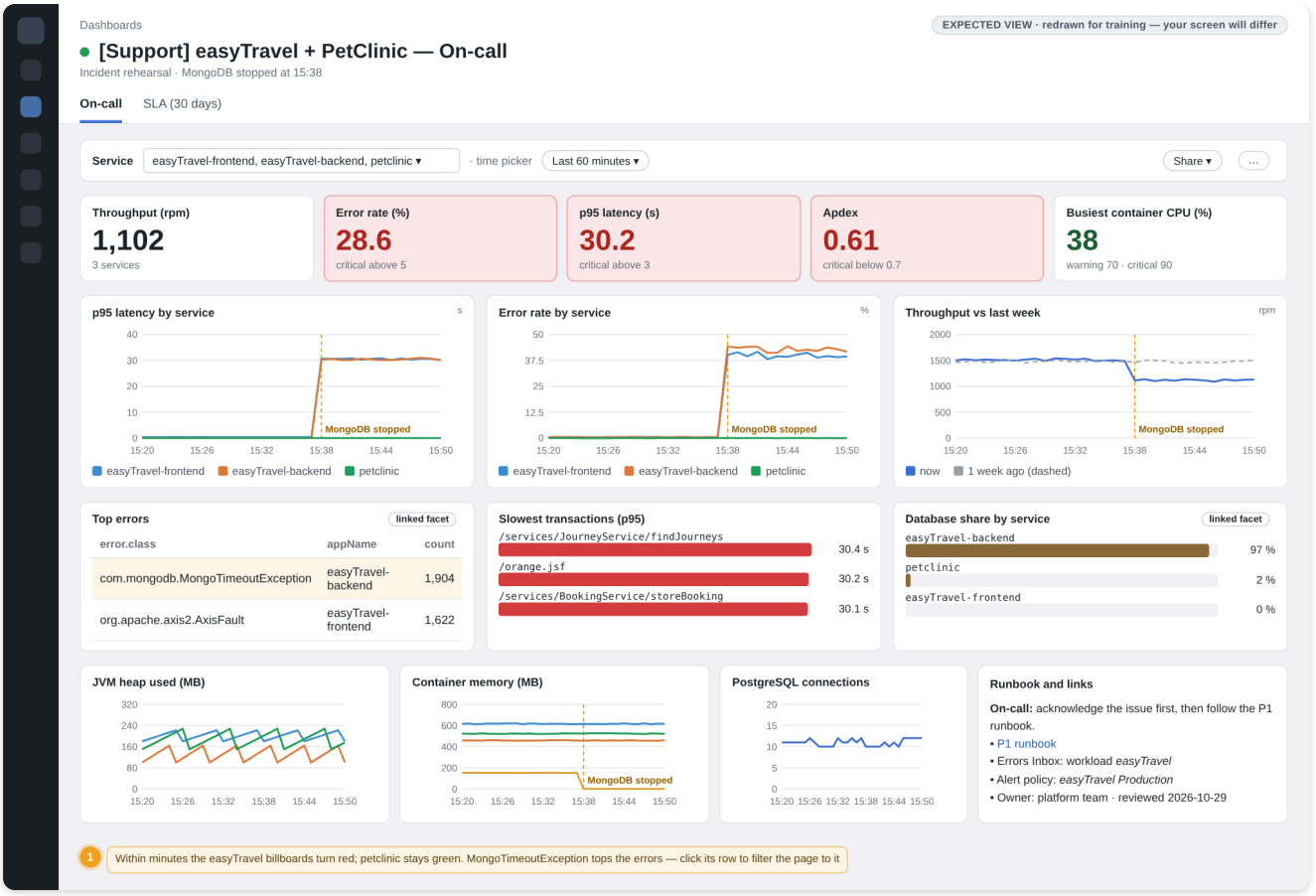
The lab only has a few days of data; the page shows the shape a real 30-day view takes.

Step 7 — Rehearse an incident with the dashboard

```
etdc stop mongodb; date +%H:%M
sleep 240
```

Watch the On-call page: the error-rate and latency billboards turn amber, then red, and the trend charts show the easyTravel lines jumping while PetClinic stays flat (set the *Service* variable to `petclinic` alone: its billboards are green). In **Top errors**, click the MongoDB timeout row: every widget whose data has `error.class` narrows to that error, showing which service and which transactions fail, and since when. Clear the filter, then restore:

```
etdc start mongodb; date +%H:%M
```



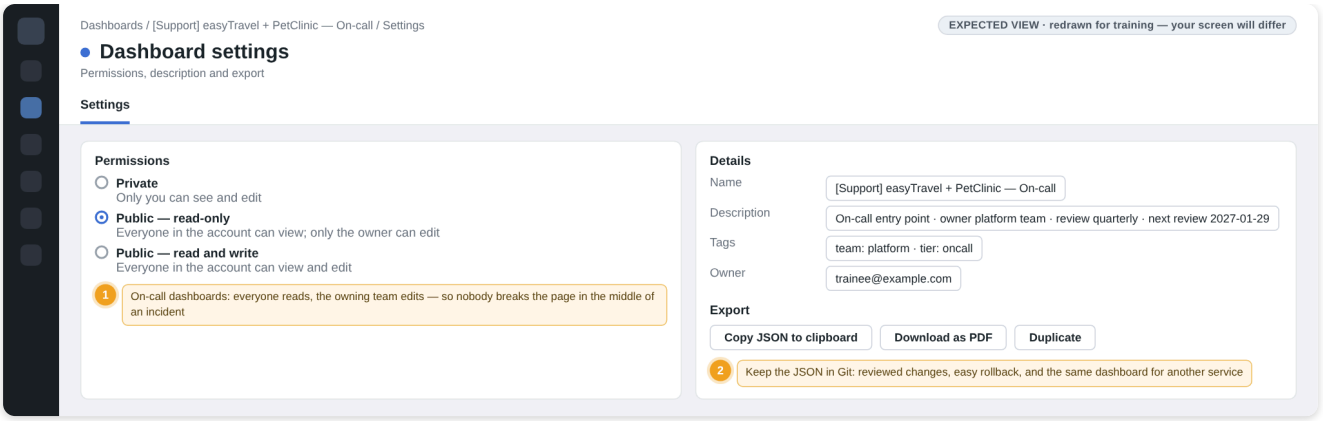
Expected view — the On-call page during the rehearsal: red billboards, the easyTravel trend lines jumping, MongoDB timeouts at the top of Top errors

Question 1. Why must the billboard thresholds match the alert condition thresholds?

Step 8 — Permissions, hygiene and JSON

1. Dashboard **settings** → **Permissions**: everyone in the account can **view**, only the owner (you, or the team) can **edit**.
2. Add a **description**: purpose, owner, review date; and a tag such as `team: platform`.
3. Copy the dashboard **JSON** (… → *Copy JSON to clipboard* or the JSON editor) and save it:

```
mkdir -p ~/nr-lab/dashboards && nano ~/nr-lab/dashboards/oncall.json   # paste, save
jq '.name, [.pages[].name], [.pages[].widgets | length]' ~/nr-lab/dashboards/oncall.json
```



Expected view — dashboard settings: view for everyone, edit for the owner, with a description that names the owner and review date

Keeping the JSON in Git lets the team review changes, roll back a broken edit, and create the same dashboard for another service by changing the variable query.

Check your work

- ☐ Two pages (On-call, SLA), a working multi-value `service` variable, and thresholds that match the alerts.
- ☐ Clicking a facet in *Top errors* or *Database share* filters the page.
- ☐ Permissions set to view-for-all; JSON saved under `~/nr-lab/dashboards`.
- ☐ MongoDB is running again.

Answer. 1 — On-call engineers read colours, not numbers. If a billboard is red at a different value than the pager fires, people either chase problems that do not exist or ignore a real one. One shared definition of "bad" keeps the alert, the dashboard and the runbook consistent.

Appendix — Reset, troubleshooting and end-of-course checklist

Reset everything

```
for p in LargeMemoryLeak MediumMemoryLeak GarbageCollectionEvery10Seconds DBSpamming
JourneySearchError500 TravellersOptionBox; do pattern off $p; done
pkill -f pcload.sh || true
etdc start mongodb backend frontend
pcdc ps --format "table {{.Name}}\t{{.Status}}"
docker ps --filter name=newrelic-infra --format "{{.Names}} {{.Status}}"
```

To remove today's additions: `pcdc down -v`, `docker rm -f newrelic-infra`, and delete `~/nr-lab/petclinic*`, `~/nr-lab/infra`.

Troubleshooting

Symptom	Check	Fix
No `SystemSample` / `ContainerSample`	`docker logs --tail 20 newrelic-infra`	License key empty (source `.env` first); on cgroup v2 hosts keep `--cgroupns=host`
JVMs page has no thread-pool charts	JMX MBeans	Standalone Tomcat registers them by default; for Spring Boot set `server.tomcat.mbeanregistry.enabled=true`
Heap query returns nothing	Step 2 discovery query	Use the exact `metricTimesliceName` your agent reports
Memory does not climb with the leak	Browser auto-completion	Type destinations repeatedly; add `MediumMemoryLeak`
Maven build fails on formatting or checks	Build output	Add `-Dcheckstyle.skip -Dspring-javaformat.skip=true` to the `mvn` command
PetClinic restarts in a loop	`pcdc logs --tail 20 petclinic`	Database not ready on first start — it retries (`restart: on-failure`); check the `POSTGRES_*` variables
No slow query samples	Agent config	`explain_threshold` 0.1 and `transaction_threshold` 0.2 in `agent/petclinic/newrelic.yml`; wait for traffic
No explain plan in the sample	Agent and database	Explain plans need PostgreSQL or MySQL and `explain_enabled: true`; use Step 8's `EXPLAIN ANALYZE` instead
No PostgreSQL entity	`docker logs --tail 50 newrelic-infra` (postgres lines)	Config file name and indentation; user and password; port 5432 published
Variable shows no values	Variable query	Query variables use their own time range — keep `SINCE 1 day ago` in the query
Linked facet does nothing	Widget settings	*Filter the current dashboard* must be on for that widget; it filters only widgets whose data has the attribute

End-of-course checklist

- ☐ All problem patterns off; MongoDB, PetClinic and the infrastructure agent running (or removed if the VM is being returned).
- ☐ Your Day 1–3 queries, runbooks and the dashboard JSON are saved outside the lab VM.
- ☐ Lab-only secrets (`.env`, the PostgreSQL monitoring password) are deleted or rotated; the user key is revoked if the account is shared.
- ☐ Alert policies, workflows and muting rules from Day 2 are disabled or deleted in shared accounts.

Sources

- New Relic — Java agent configuration (transaction tracer, record_sql, explain plans)
- New Relic — Docker container monitoring (ContainerSample)
- New Relic — PostgreSQL integration
- New Relic — Dashboard template variables
- Spring PetClinic (GitHub)
- PostgreSQL — Operator classes and text_pattern_ops